Capítulo 1. Deep Generative Models

Trabajo de Fin de Grado

Arturo Olivares Martos

Doble Grado en Ingeniería Informática y Matemáticas
Universidad de Granada

7 de julio de 2026



Índice

1. Objetivo de un DGM
2. Función de pérdida
3. Camino a obtener p_ϕ
4. DGM destacados



Distribución de los datos

Los datos de alta dimensionalidad con los que trabajemos siguien una distribución de probabilidad:

$$p_{\text{data}}$$



Distribución de los datos

Los datos de alta dimensionalidad con los que trabajemos siguien una distribución de probabilidad:

$$p_{\text{data}}$$

- Los datos reales tendrán alta probabilidad, mientras que los irreales (ruido, etc.) tendrán probabilidad prácticamente nula.



Distribución de los datos

Los datos de alta dimensionalidad con los que trabajemos siguien una distribución de probabilidad:

$$p_{\text{data}}$$

- Los datos reales tendrán alta probabilidad, mientras que los irreales (ruido, etc.) tendrán probabilidad prácticamente nula.
- Esta distribución es *desconocida* (tan solo tenemos acceso a una muestra de datos).



Distribución del modelo

El DGM emplea una DNN con parámetros ϕ para obtener una distribución de probabilidad

$$p_\phi$$

de manera que $p_\phi \approx p_{\text{data}}$.



Distribución del modelo

El DGM emplea una DNN con parámetros ϕ para obtener una distribución de probabilidad

$$p_\phi$$

de manera que $p_\phi \approx p_{\text{data}}$.

Objetivo de un DGM

Aprender la distribución p_{data} a partir de una muestra de datos, para poder generar nuevas muestras que sigan la misma distribución.



Generación de nuevas muestras

Una vez aprendido el modelo, podemos generar nuevas muestras de datos siguiendo la distribución p_ϕ:

$$x \sim p_\phi \approx p_{\text{data}}$$



Generación de nuevas muestras

Una vez aprendido el modelo, podemos generar nuevas muestras de datos siguiendo la distribución p_ϕ:

$$x \sim p_\phi \approx p_{\text{data}}$$

Esta generación debe cumplir:



Generación de nuevas muestras

Una vez aprendido el modelo, podemos generar nuevas muestras de datos siguiendo la distribución p_ϕ:

$$x \sim p_\phi \approx p_{\text{data}}$$

Esta generación debe cumplir:

- Generación realista (Test de Lovelance)



Generación de nuevas muestras

Una vez aprendido el modelo, podemos generar nuevas muestras de datos siguiendo la distribución p_ϕ:

$$x \sim p_\phi \approx p_{\text{data}}$$

Esta generación debe cumplir:

- Generación realista (Test de Lovelance)
- Generación controlada



Índice

1 Objetivo de un DGM

2 Función de pérdida

3 Camino a obtener p_ϕ

4 DGM destacados



Función de pérdida

Divergencia

Una divergencia entre dos distribuciones de probabilidad p_{data} y p_ϕ es una función $\mathcal{D}(p_{\text{data}}, p_\phi)$ que cumple:

- No negatividad: $\mathcal{D}(p_{\text{data}}, p_\phi) \geq 0$
- Identidad: $\mathcal{D}(p_{\text{data}}, p_\phi) = 0 \iff p_{\text{data}} = p_\phi$ c.p.d.



Función de pérdida

Divergencia

Una divergencia entre dos distribuciones de probabilidad p_{data} y p_ϕ es una función $\mathcal{D}(p_{\text{data}}, p_\phi)$ que cumple:

- No negatividad: $\mathcal{D}(p_{\text{data}}, p_\phi) \geq 0$
- Identidad: $\mathcal{D}(p_{\text{data}}, p_\phi) = 0 \iff p_{\text{data}} = p_\phi$ c.p.d.

Entrenar la DNN consiste en obtener los parámetros ϕ que minimicen la función de pérdida. Hay varias funciones de pérdida que se pueden emplear.



Divergencia KL

Divergencia KL Directa

La divergencia KL directa entre dos distribuciones de probabilidad p_{data} y p_ϕ es:

$$\mathcal{D}_{\text{KL}}(p_{\text{data}}||p_\phi) := \int p_{\text{data}}(x) \log \frac{p_{\text{data}}(x)}{p_\phi(x)} dx$$

donde se ha tomado la primera distribución p_{data} como referencia.



Divergencia KL

Divergencia KL Directa

La divergencia KL directa entre dos distribuciones de probabilidad p_{data} y p_ϕ es:

$$\mathcal{D}_{\text{KL}}(p_{\text{data}}||p_\phi) := \int p_{\text{data}}(x) \log \frac{p_{\text{data}}(x)}{p_\phi(x)} dx$$

donde se ha tomado la primera distribución p_{data} como referencia.

Emplear la divergencia KL fuerza la cobertura del modelo; es decir, toda muestra x con $p_{\text{data}}(x) > 0$ debe tener $p_\phi(x) > 0$ (para no dividir entre cero).

$$p_{\text{data}}(x) > 0 \implies p_\phi(x) > 0 \qquad \forall x$$



Divergencia KL

Puesto que no conocemos la distribución p_{data}, no podemos calcular de forma directa la divergencia KL.

$$\begin{aligned}\mathcal{D}_{\text{KL}}(p_{\text{data}}||p_\phi) &:= \int p_{\text{data}}(x) \log \frac{p_{\text{data}}(x)}{p_\phi(x)} dx \\ &= \mathbb{E}_{x\sim p_{\text{data}}} \left[\log \frac{p_{\text{data}}(x)}{p_\phi(x)}\right] \\ &= \mathbb{E}_{x\sim p_{\text{data}}} \left[\log p_{\text{data}}(x) - \log p_\phi(x)\right] \\ &= \mathbb{E}_{x\sim p_{\text{data}}} \left[\log p_{\text{data}}(x)\right] - \mathbb{E}_{x\sim p_{\text{data}}} \left[\log p_\phi(x)\right]\end{aligned}$$



Divergencia KL

Entropía

La entropía de una distribución de probabilidad p_{data} es:

$$\mathcal{H}(p_{\text{data}}) := -\int p_{\text{data}}(x) \log p_{\text{data}}(x) dx = -\mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\text{data}}(x)]$$



Divergencia KL

Entropía

La entropía de una distribución de probabilidad p_{data} es:

$$\mathcal{H}(p_{\text{data}}) := -\int p_{\text{data}}(x) \log p_{\text{data}}(x) dx = -\mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\text{data}}(x)]$$

Cálculo de la divergencia KL

$$\mathcal{D}_{\text{KL}}(p_{\text{data}} || p_\phi) = -\mathcal{H}(p_{\text{data}}) - \mathbb{E}_{x \sim p_{\text{data}}} [\log p_\phi(x)]$$



Divergencia KL

Entropía

La entropía de una distribución de probabilidad p_{data} es:

$$\mathcal{H}(p_{\text{data}}) := -\int p_{\text{data}}(x) \log p_{\text{data}}(x) dx = -\mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\text{data}}(x)]$$

Cálculo de la divergencia KL

$$\mathcal{D}_{\text{KL}}(p_{\text{data}} || p_\phi) = -\mathcal{H}(p_{\text{data}}) - \mathbb{E}_{x \sim p_{\text{data}}} [\log p_\phi(x)]$$

¡$\mathcal{H}(p_{\text{data}})$ no depende de los parámetros ϕ!



Minimización de la divergencia KL

Lema (Minimizar KL $\iff$ MLE (Maximum Likelihood Estimation))

$$\min_{\phi} \mathcal{D}_{KL}(p_{data}||p_\phi) \iff \max_{\phi} \mathbb{E}_{x\sim p_{data}} [\log p_\phi(x)]$$



Minimización de la divergencia KL

Lema (Minimizar KL $\iff$ MLE (Maximum Likelihood Estimation))

$$\min_\phi \mathcal{D}_{KL}(p_{data}||p_\phi) \iff \max_\phi \mathbb{E}_{x\sim p_{data}}[\log p_\phi(x)]$$

Estimando mediante Monte Carlo la esperanza (y teniendo en cuenta que se prefiere minimizar la función de pérdida), obtenemos la función de pérdida empleada en la práctica:

$$\hat{\mathcal{L}}_{\text{MLE}}(\phi) = -\frac{1}{N}\sum_{i=1}^{N}\log p_\phi(x_i) = -\frac{1}{N}\log\prod_{i=1}^{N} p_\phi(x_i) \qquad x_i \sim p_{\text{data}}$$



Divergencia de Fisher

Divergencia de Fisher

La divergencia de Fisher entre dos distribuciones de probabilidad p_{data} y p_ϕ es:

$$\mathcal{D}_{\text{F}}(p_{\text{data}}||p_\phi) := E_{x\sim p_{\text{data}}}\left[\|\nabla_x \log p_{\text{data}}(x) - \nabla_x \log p_\phi(x)\|_2^2\right]$$

donde se ha tomado la primera distribución p_{data} como referencia.



f-Divergencias

f-Divergencia

Sea $f : \mathbb{R}^+ \to \mathbb{R}$ una función convexa con $f(1) = 0$. La f-divergencia entre dos distribuciones de probabilidad p y q es:

$$\mathcal{D}_f(p||q) := \int q(x) f\left(\frac{p(x)}{q(x)}\right) dx$$



f-Divergencias

Casos particulares

- Divergencia KL directa $\mathcal{D}_{\mathrm{KL}}(p||q)$: $f(u) = u \log u$
- Divergencia de Jensen-Shannon $\mathcal{D}_{\mathrm{JS}}(p||q)$:

$$f(u) = \frac{1}{2}\left[u \log u - (u+1)\log\left(\frac{u+1}{2}\right)\right]$$

- Variación Total $\mathcal{D}_{\mathrm{TV}}(p||q)$: $f(u) = \frac{1}{2}|u-1|$



f-Divergencias

Casos particulares

- Divergencia KL directa $\mathcal{D}_{\mathrm{KL}}(p||q)$: $f(u) = u \log u$
- Divergencia de Jensen-Shannon $\mathcal{D}_{\mathrm{JS}}(p||q)$:

$$f(u) = \frac{1}{2}\left[u \log u - (u+1)\log\left(\frac{u+1}{2}\right)\right]$$

- Variación Total $\mathcal{D}_{\mathrm{TV}}(p||q)$: $f(u) = \frac{1}{2}|u-1|$

$$\mathcal{D}_{\mathrm{JS}}(p||q) = \frac{1}{2}\mathcal{D}_{\mathrm{KL}}\left(p||\frac{1}{2}(p+q)\right) + \frac{1}{2}\mathcal{D}_{\mathrm{KL}}\left(q||\frac{1}{2}(p+q)\right),$$

$$\mathcal{D}_{\mathrm{TV}}(p,q) = \frac{1}{2}\int |p-q|dx = \sup_{A\subset\mathbb{R}^D} |p(A) - q(A)|.$$



Índice

1. Objetivo de un DGM
2. Función de pérdida
3. Camino a obtener p_ϕ
4. DGM destacados



Camino a obtener p_ϕ

Para que la función p_ϕ buscada sea realmente una función de densidad de probabilidad, debe cumplir:

1. No negatividad:
$$p_\phi(x) \geq 0 \qquad \forall x$$
2. Normalización:
$$\int p_\phi(x)dx = 1$$



Camino a obtener p_ϕ

Para que la función p_ϕ buscada sea realmente una función de densidad de probabilidad, debe cumplir:

1. No negatividad:
$$p_\phi(x) \geq 0 \qquad \forall x$$
2. Normalización:
$$\int p_\phi(x)dx = 1$$

La DNN puede producir $E_\phi(x) \in \mathbb{R}$ para cada entrada x, pero hemos de realizar varias transformaciones para obtener $p_\phi(x)$.



No negatividad

Para garantizar la no negatividad, podemos emplear cualquier función positiva a $E_\phi(x)$, como $|E_\phi(x)|$ o $(E_\phi(x))^2$.



No negatividad

Para garantizar la no negatividad, podemos emplear cualquier función positiva a $E_\phi(x)$, como $|E_\phi(x)|$ o $(E_\phi(x))^2$.

En la práctica, se emplea la función exponencial:

$$\tilde{p}_\phi(x) = \exp(E_\phi(x)) \geq 0 \qquad \forall x$$



Normalización

Para garantizar la normalización, podemos dividir la función $\tilde{p}_\phi(x)$ entre su integral:

$$p_\phi(x) = \frac{\tilde{p}_\phi(x)}{\int \tilde{p}_\phi(x')dx'} = \frac{\exp(E_\phi(x))}{Z(\phi)}$$

donde se ha definido la *constante de normalización* o *función de partición*:

$$Z(\phi) := \int \tilde{p}_\phi(x')dx' = \int \exp(E_\phi(x'))dx'$$



Normalización

Para garantizar la normalización, podemos dividir la función $\tilde{p}_\phi(x)$ entre su integral:

$$p_\phi(x) = \frac{\tilde{p}_\phi(x)}{\int \tilde{p}_\phi(x')dx'} = \frac{\exp(E_\phi(x))}{Z(\phi)}$$

donde se ha definido la *constante de normalización* o *función de partición*:

$$Z(\phi) := \int \tilde{p}_\phi(x')dx' = \int \exp(E_\phi(x'))dx'$$

Esta integral es muy costosa de calcular, por lo que distintos tipos de DGM emplean distintas técnicas para aproximarla o evitarla.



Índice

1. Objetivo de un DGM
2. Función de pérdida
3. Camino a obtener p_ϕ
4. DGM destacados



DGM destacados

Distintos DGM emplean distintas técnicas para aproximar o evitar la integral de normalización, haciendo distintos trade-offs entre:



DGM destacados

Distintos DGM emplean distintas técnicas para aproximar o evitar la integral de normalización, haciendo distintos trade-offs entre:

- Tractabilidad: qué tan fácil es calcular la función de densidad de probabilidad $p_\phi(x)$ para una muestra x dada.



DGM destacados

Distintos DGM emplean distintas técnicas para aproximar o evitar la integral de normalización, haciendo distintos trade-offs entre:

- Tractabilidad: qué tan fácil es calcular la función de densidad de probabilidad $p_\phi(x)$ para una muestra x dada.
- Expresividad: qué tan bien puede aproximar la verdadera complejidad de los datos.



DGM destacados

Distintos DGM emplean distintas técnicas para aproximar o evitar la integral de normalización, haciendo distintos trade-offs entre:

- Tractabilidad: qué tan fácil es calcular la función de densidad de probabilidad $p_\phi(x)$ para una muestra x dada.
- Expresividad: qué tan bien puede aproximar la verdadera complejidad de los datos.
- Eficiencia de entrenamiento: qué tan rápido se puede entrenar el modelo.



Modelos Basados en Energía (EBM)

Modelo Basado en Energía

Un Modelo Basado en Energía (EBM) es un DGM que emplea una DNN para obtener una función de energía $E_\phi(x)$ *(donde se asigna baja energía a los datos más probables)*, y define:

$$p_\phi(x) = \frac{\exp(-E_\phi(x))}{Z(\phi)} \qquad \text{con} \qquad Z(\phi) := \int \exp(-E_\phi(x'))dx'$$

[1] Al aplicar el logaritmo al cociente y convertirlo en una resta, la derivada de la constante se anula, evitando calcularla.

Modelos Basados en Energía (EBM)

Modelo Basado en Energía

Un Modelo Basado en Energía (EBM) es un DGM que emplea una DNN para obtener una función de energía $E_\phi(x)$ *(donde se asigna baja energía a los datos más probables)*, y define:

$$p_\phi(x) = \frac{\exp(-E_\phi(x))}{Z(\phi)} \qquad \text{con} \qquad Z(\phi) := \int \exp(-E_\phi(x'))dx'$$

Los Modelos de Difusión generarán muestras usando *el gradiente del logaritmo de la función de densidad de probabilidad*[1], evitando así calcular $Z(\phi)$.

[1] Al aplicar el logaritmo al cociente y convertirlo en una resta, la derivada de la constante se anula, evitando calcularla.

Modelos Autoregresivos (AR)

Modelo Autoregresivo

Un Modelo Autoregresivo (AR) es un DGM que descompone la función de densidad de probabilidad $p_\phi(x)$ en una cadena de distribuciones condicionales:

$$p_\phi(x) = \prod_{i=1}^{D} p_\phi(x_i | x_{<i})$$

donde $x = (x_1, x_2, \ldots, x_D)$ y $x_{<i} = (x_1, x_2, \ldots, x_{i-1})$.

[2] Esto se puede hacer porque son unidimensionales.

Modelos Autoregresivos (AR)

Modelo Autoregresivo

Un Modelo Autoregresivo (AR) es un DGM que descompone la función de densidad de probabilidad $p_\phi(x)$ en una cadena de distribuciones condicionales:

$$p_\phi(x) = \prod_{i=1}^{D} p_\phi(x_i|x_{<i})$$

donde $x = (x_1, x_2, \ldots, x_D)$ y $x_{<i} = (x_1, x_2, \ldots, x_{i-1})$.

Cada distribución condicional $p_\phi(x_i|x_{<i})$ se modela mediante una DNN y se normaliza por construcción[2] (empleando determinadas funciones de activación como *softmax* o *Gaussiana parametrizada*), evitando así calcular $Z(\phi)$.

[2] Esto se puede hacer porque son unidimensionales.

Autoencoders Variacionales (VAE)

Autoencoder Variacional

Un Autoencoder Variacional (VAE) es un DGM que emplea variables latentes z que representan estructuras de alto nivel de los datos.



Autoencoders Variacionales (VAE)

Autoencoder Variacional

Un Autoencoder Variacional (VAE) es un DGM que emplea variables latentes z que representan estructuras de alto nivel de los datos.Emplea dos DNN:

- Un *encoder* $q_\theta(z|x)$ que aproxima la distribución posterior de las variables latentes dado un dato x.



Autoencoders Variacionales (VAE)

Autoencoder Variacional

Un Autoencoder Variacional (VAE) es un DGM que emplea variables latentes z que representan estructuras de alto nivel de los datos.Emplea dos DNN:

- Un *encoder* $q_\theta(z|x)$ que aproxima la distribución posterior de las variables latentes dado un dato x.
- Un *decoder* $p_\phi(x|z)$ que reconstruye el dato x a partir de las variables latentes z.



Autoencoders Variacionales (VAE)

En vez de maximizar la probabilidad logarítmica de los datos, el VAE maximiza una cota inferior de la misma, llamada *Evidence Lower Bound (ELBO)*:

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; x) = \mathbb{E}_{q_\theta(z|x)}[\log p_\phi(x|z)] - \mathcal{D}_{\text{KL}}(q_\theta(z|x) \parallel p_{\text{prior}}(z))$$



Autoencoders Variacionales (VAE)

En vez de maximizar la probabilidad logarítmica de los datos, el VAE maximiza una cota inferior de la misma, llamada *Evidence Lower Bound (ELBO)*:

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; x) = \mathbb{E}_{q_\theta(z|x)}[\log p_\phi(x|z)] - \mathcal{D}_{\text{KL}}(q_\theta(z|x) \parallel p_{\text{prior}}(z))$$

- El primer término representa la reconstrucción correcta del dato x a partir de las variables latentes z.



Autoencoders Variacionales (VAE)

En vez de maximizar la probabilidad logarítmica de los datos, el VAE maximiza una cota inferior de la misma, llamada *Evidence Lower Bound (ELBO)*:

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; x) = \mathbb{E}_{q_\theta(z|x)}[\log p_\phi(x|z)] - \mathcal{D}_{\text{KL}}(q_\theta(z|x) \parallel p_{\text{prior}}(z))$$

- El primer término representa la reconstrucción correcta del dato x a partir de las variables latentes z.
- El segundo término busca regularizar la distribución posterior de las variables latentes para que se acerque a una distribución previa simple (como una Gaussiana estándar), evitando así el *overfitting*.



Normalizing Flows (NF)

Flujo Normal

Un Normalizing Flow (NF) es un DGM que transforma una distribución simple de variables latentes $z \sim p_{\text{prior}}(z)$ (como una Gaussiana estándar) en la distribución de los datos $x \sim p_\phi(x)$ mediante una función (o cadena de funciones) invertible y diferenciable f_ϕ:

$$x = f_\phi(z) \qquad \text{y} \qquad z = f_\phi^{-1}(x)$$



Normalizing Flows (NF)

La función de densidad de probabilidad $p_\phi(x)$ se obtiene mediante el cambio de variable:

$$p_\phi(x) = p_{\text{prior}}(f_\phi^{-1}(x)) \left| \det \left(\frac{\partial f_\phi^{-1}(x)}{\partial x} \right) \right|$$



Normalizing Flows (NF)

La función de densidad de probabilidad $p_\phi(x)$ se obtiene mediante el cambio de variable:

$$p_\phi(x) = p_{\text{prior}}(f_\phi^{-1}(x)) \left| \det \left(\frac{\partial f_\phi^{-1}(x)}{\partial x} \right) \right|$$

Al estar $p_{\text{prior}}(z)$ normalizada, también lo estará $p_\phi(x)$ automáticamente, evitando así calcular $Z(\phi)$.

$$\begin{aligned} \int p_\phi(x)dx &= \int p_{\text{prior}}(f_\phi^{-1}(x)) \left| \det \left(\frac{\partial f_\phi^{-1}(x)}{\partial x} \right) \right| dx \\ &= \int p_{\text{prior}}(z)dz = 1 \end{aligned}$$



Generative Adversarial Networks (GAN)

Red Generativa Adversaria

Una GAN (Generative Adversarial Network) es un DGM que emplea dos DNN:

- Un *generador* G_ϕ que busca transformar ruido simple $z \sim p_{\text{prior}}(z)$ en muestras realistas $G_\phi(z)$.



Generative Adversarial Networks (GAN)

Red Generativa Adversaria

Una GAN (Generative Adversarial Network) es un DGM que emplea dos DNN:

- Un *generador* G_ϕ que busca transformar ruido simple $z \sim p_{\text{prior}}(z)$ en muestras realistas $G_\phi(z)$.
- Un *discriminador* D_θ que busca distinguir entre muestras reales $x \sim p_{\text{data}}$ y muestras generadas $G_\phi(z)$.



Generative Adversarial Networks (GAN)

Red Generativa Adversaria

Una GAN (Generative Adversarial Network) es un DGM que emplea dos DNN:

- Un *generador* G_ϕ que busca transformar ruido simple $z \sim p_{\text{prior}}(z)$ en muestras realistas $G_\phi(z)$.
- Un *discriminador* D_θ que busca distinguir entre muestras reales $x \sim p_{\text{data}}$ y muestras generadas $G_\phi(z)$.

La función objetivo de la GAN es un juego de suma cero entre el generador y el discriminador:

$$\min_\phi \max_\theta \underbrace{\mathbb{E}_{x\sim p_{\text{data}}}[\log D_\theta(x)]}_{\text{Reales}} + \underbrace{\mathbb{E}_{z\sim p_{\text{prior}}}[\log(1 - D_\theta(G_\phi(z)))]}_{\text{Generadas}}$$



Generative Adversarial Networks (GAN)

Red Generativa Adversaria

Una GAN (Generative Adversarial Network) es un DGM que emplea dos DNN:

- Un *generador* G_ϕ que busca transformar ruido simple $z \sim p_{\text{prior}}(z)$ en muestras realistas $G_\phi(z)$.
- Un *discriminador* D_θ que busca distinguir entre muestras reales $x \sim p_{\text{data}}$ y muestras generadas $G_\phi(z)$.

La función objetivo de la GAN es un juego de suma cero entre el generador y el discriminador:

$$\min_\phi \max_\theta \underbrace{\mathbb{E}_{x\sim p_{\text{data}}}[\log D_\theta(x)]}_{\text{Reales}} + \underbrace{\mathbb{E}_{z\sim p_{\text{prior}}}[\log(1 - D_\theta(G_\phi(z)))]}_{\text{Generadas}}$$

¡No calcula ni estima ninguna probabilidad!

